Lab 2: Experiments on microgpt, with an agent

CMSC 35200 · 100 points · Due Tuesday, October 20, 2026 at 11:59 pm (Chicago time)

Submit via Classroom50

Released October 11, 2026

Overview

This lab uses Andrej Karpathy's *microgpt* (Karpathy 2026a,b), a 200-line dependency-free Python program that trains and samples a GPT, as an experimental testbed. You will work **with a coding agent** (Claude Code or an equivalent tool that can edit and run files). The agent does the implementation; you do the prediction, the experimental design, and the explanation. The lab is graded on evidence that *you* understood what happened, not on the quality of the agent's code.

The lab has ten sections worth 10 points each. Every section follows the same pattern:

1. **Predict.** Before the agent runs anything for that section, you write your prediction in `predictions.md`. Predictions are graded on whether they are specific and were made before the result, not on whether they were right.
2. **Delegate.** You instruct the agent. The transcript of that interaction is a submitted artifact.
3. **Explain.** You write the analysis in the report in your own words.

Materials. The lecture notes, slides and study guide for microgpt; the post and gist linked above. Baseline settings are those of the gist: seed 42, $d = 16$, 4 heads, 1 layer, 1,000 steps, Adam with linear decay.

Ground rules.

- Keep `random.seed(42)` and the baseline hyperparameters unless a section tells you to change them. State every deviation in the report.
- A pure-Python run takes one to four minutes. For sweeps, you may reduce `num_steps`, but every arm of a comparison must use the same number of steps and the same seed.
- Do not let the agent choose experimental parameters that a section assigns to you (tolerances, step sizes, learning-rate grids, protocols). Those choices are the graded content.
- Report every number with the setting that produced it. A table without its settings earns no credit.

What to submit

Upload to Classroom50 **two items**: the report `report.pdf` by itself (so it can be annotated), and one archive `lab2_cnetid.zip` with the layout below. Both are required; the zip must also contain the report.

Path in the zip	Contents
`README.md`	Your name, CNetID, the agent and model you used, how to rerun each section (one command per section), and total wall-clock time spent on runs.
`report.pdf`	The written report: one subsection per lab section, in order, each containing the required tables, figures and explanations. Target 8–14 pages.
`predictions.md`	Your predictions for all ten sections, each dated and time-stamped *before* the corresponding runs. Commit history or file timestamps may be checked.
`code/`	Every script the agent produced, one subfolder per section (`code/s01/` ... `code/s10/`), plus the unmodified `microgpt.py` for reference. Each script must run from the command line.
`results/`	Raw outputs: CSV files of losses, sweep tables, generated samples, gradient-check output. One subfolder per section.
`figures/`	Every plot referenced in the report, as PDF or PNG, named `sNN_description`.
`transcripts/`	The full agent transcripts, one file per section (`transcripts/s01.md` ...). Export or copy the entire conversation, including your prompts. Transcripts that have been edited for content will be treated as missing.

Grading of each section. Within each 10-point section: **3 points** for a specific, timely prediction; **4 points** for correct, reproducible artifacts (code runs, numbers match settings, figures labelled); **3 points** for the explanation. Explanations are graded for mechanism, not length: one correct sentence about *why* outranks a page of description.

Agent use and integrity. Using an agent is required, and everything the agent wrote must be in `code/` with its transcript in `transcripts/`. The report and predictions must be written by you. You may ask the agent to check your arithmetic or your reasoning, but a report that reads as agent output, or that contains explanations not supported by your transcripts, will be graded as such. Collaboration between students on experimental design is fine; shared code, results or text is not.

1 Reproduce, then instrument the loss (10 points)

Goal. Know what the printed loss measures and why it fluctuates.

Predict first (write this in `predictions.md` before the agent runs anything)

The loss of a uniform guess over 27 tokens (show the arithmetic); whether the step-1 loss will be above or below it; whether the per-step loss will decrease smoothly; and whether a held-out loss will diverge from the training loss within 1,000 steps.

Ask the agent to. Run the unmodified script and save every step's loss, the document, and its length to `results/s01/loss.csv`. Plot the raw loss, a trailing 50-step mean, and a horizontal line at your predicted uniform baseline. Then add a held-out split of 10% of the names, evaluated every 100 steps with no `backward()` call, and add it to the plot.

Then you. Sort the last 100 training steps by loss and inspect the names at both extremes. Explain why the raw curve is noisy and why the final smoothed value differs from the last printed value. Convert the final smoothed loss to perplexity and to bits per character *by hand*, showing the arithmetic, then have the agent check it. Explain why no train/held-out gap appears.

Artifacts for this section

`code/s01/`, `results/s01/loss.csv`, `results/s01/heldout.csv`, `figures/s01_loss.pdf`; in the report: the plot, a table of the five lowest- and five highest-loss steps from the last 100 with their names, your hand calculation, and the two explanations.

Points (10)

3 prediction · 4 artifacts (CSV complete, plot labelled, held-out evaluated without gradients) · 3 explanation (noise mechanism, smoothing, no-gap reasoning).

2 Bug hunt with a gradient checker you design (10 points)

Goal. Understand `backward()` well enough to detect when it is wrong.

Predict first (write this in `predictions.md` before the agent runs anything)

For each of the four candidate bugs below, what symptom you expect in (a) a finite-difference gradient check and (b) the loss curve.

Ask the agent to. Produce three copies of the script, each with exactly one silent bug in the autograd engine chosen from: `child.grad = ...` instead of `+=`; no `p.grad = 0` reset; iterating `topo` forward instead of reversed; a wrong local gradient in `__pow__`. The agent must not tell you which copy has which bug (instruct it to record the mapping in a sealed file `results/s02/answer_key.txt` that you open only after submitting your diagnosis in the report).

Then you. Write the specification of a finite-difference gradient checker: which parameters to sample and how many, the step size h, and the comparison metric (relative error, with the formula). The agent implements it to your spec. Run it on the three copies and the original and diagnose each copy from checker output and

loss curves *before* reading the diffs. Then show, with numbers, what happens to the checker when h is 100× too large and 100× too small, and explain both failure modes.

Artifacts for this section

`code/s02/` (checker plus the three buggy copies), `results/s02/` (checker output for all four scripts, loss CSVs, the sealed answer key), in the report: your checker spec, a diagnosis table (copy → bug → evidence), the h-sensitivity table, and the explanation.

Points (10)

3 prediction · 4 artifacts (spec implemented as written, all four scripts checked, diagnoses recorded before the key) · 3 explanation (correct diagnoses with mechanism; both h failure modes).

3 The bigram baseline and what attention adds (10 points)

Goal. Quantify what the Transformer buys over counting.

Predict first (write this in `predictions.md` before the agent runs anything)

The per-token loss of an add-one-smoothed bigram model on the names relative to microgpt's ≈ 2.3; the same for a trigram; and whether microgpt with `block_size = 2` will land nearer the bigram or the trigram.

Ask the agent to. Implement a bigram model (a 27 × 27 table of next-token counts over `BOS`-wrapped names, add-one smoothing, row normalisation) and a trigram model, compute each one's average per-token loss over the full dataset, and sample 20 names from each. Then set microgpt's `block_size` to 2 and retrain with baseline settings.

Then you. Fill in a four-row table (bigram, trigram, microgpt with block size 2, microgpt baseline) with loss and parameter or table size. Explain what information each row has that the previous row lacks, and why the block-size-2 result landed where it did. Comment on sample quality with examples.

Artifacts for this section

`code/s03/`, `results/s03/` (loss values, 20 samples per model), in the report: the table, samples, and explanation.

Points (10)

3 prediction · 4 artifacts (correct smoothing and `BOS` handling, losses on the full dataset, block-size run) · 3 explanation.

4 The ablation ladder (10 points)

Goal. Learn what each architectural piece contributes by removing it.

Predict first (write this in `predictions.md` before the agent runs anything)

Rank the five ablations from least to most damaging, with one sentence of reasoning each: (a) remove the position embedding `wpe`; (b) remove both residual additions; (c) remove the first `rmsnorm` after the embeddings; (d) one head of width 16 instead of four of width 4; (e) remove the MLP block.

Ask the agent to. Implement each ablation as a command-line flag in one copy of the script, run all five plus the baseline with the same seed and step count, and report the trailing 100-step mean loss and the parameter count for each.

Then you. Compare the measured ranking with your predicted ranking. For every misplacement, write one sentence on what you had misunderstood. Address (c) specifically in light of the source comment that it is "not redundant due to backward pass via the residual connection", and (d) in light of the fact that it changes no parameter count.

Artifacts for this section

`code/s04/` (one script, six flags), `results/s04/ablations.csv`, `figures/s04_curves.pdf` (six smoothed curves on one axis), in the report: predicted vs measured ranking table and the explanations.

Points (10)

3 prediction (full ranking with reasons) · 4 artifacts (six runs, equal settings, parameter counts correct) · 3 explanation.

5 Look inside attention (10 points)

Goal. Test the post's suggestion that a head might ask "what vowels appeared recently?"

Predict first (write this in `predictions.md` before the agent runs anything)

What pattern you expect each of the four heads to show after training (for example previous-token, first-token, diffuse), and one hypothesis about one head that can be tested quantitatively.

Ask the agent to. After training, record the attention weights of every head at every position for a given input name, and render one heatmap per head (query positions as rows, cached key positions as columns) for five names you choose, including one long name and one with a repeated letter. Then implement the quantitative test you specify: for example, the mean weight placed on the most recent vowel versus all other positions, averaged over 500 training names.

Then you. Describe each head's pattern in words *before* reading any interpretation the agent offers. State your hypothesis, the statistic that tests it, the result, and whether the hypothesis survives. If it does not, say what the head appears to do instead.

Artifacts for this section

`code/s05/`, `figures/s05_head{0..3}_name.pdf` (20 heatmaps, or a grid), `results/s05/hypothesis_test.csv`, in the report: a 4-row table (head → your description), the hypothesis test with its statistic, and the conclusion.

Points (10)

3 prediction · 4 artifacts (correct weights captured post-training, heatmaps labelled, test implemented as specified) · 3 explanation (descriptions precede interpretation; conclusion follows from the statistic).

6 Port it to tensors and prove equivalence (10 points)

Goal. Show that the scalar graph and a vectorised implementation are the same algorithm.

Predict first (write this in `predictions.md` before the agent runs anything)

The tolerance at which logits and gradients from the two implementations should agree, with justification from floating-point precision; and the expected speed-up of one training step, with reasoning from the node counts in the notes.

Ask the agent to. Write a PyTorch version that processes a whole document in one forward pass with a causal mask, initialised with the exact weights copied out of the scalar version's `state_dict` after seeding. Compare logits at every position for the first training document and the gradient of every parameter after one backward pass. Time one training step in each implementation.

Then you. Report the maximum absolute and relative discrepancies against your tolerance. Answer: where did the KV cache go in the tensor version, and why does it need a mask when microgpt does not? Explain the measured speed ratio.

Artifacts for this section

`code/s06/` (the PyTorch port and the comparison script), `results/s06/equivalence.txt` (discrepancies and timings), in the report: tolerance with justification, discrepancy table, timing table, and the two explanations.

Points (10)

3 prediction · 4 artifacts (identical init, logits and *gradients* compared, timings) · 3 explanation (cache/mask correspondence; speed ratio).

7 Optimizer bake-off with a fair protocol (10 points)

Goal. Understand what each term of Adam does.

Predict first (write this in `predictions.md` before the agent runs anything)

Which optimizer will reach the lowest loss at the fixed budget; what the first 20 steps of Adam without bias correction will look like and why; and which parameters will receive the largest Adam steps relative to their raw gradients.

Ask the agent to. Implement plain SGD, SGD with momentum, RMSProp, Adam without bias correction, and Adam, each as a flag, and run the sweep exactly as you specify. Afterwards print, for ten parameters you name (include embedding rows for `q`, `x` and `a`), the ratio of the Adam update to the raw gradient at steps 1, 10 and 100.

Then you. Write the protocol *first*: identical number of documents per arm, a learning-rate grid of at least five values per arm (state it), the metric (trailing 100-step mean), and how you will present best-of-grid results. Plot best loss per optimizer and the best curve from each arm. Explain the early behaviour without bias correction, and why rarely used embedding rows get larger steps.

Artifacts for this section

`code/s07/`, `results/s07/sweep.csv` (every run with its settings), `results/s07/update_ratios.csv`, `figures/s07_sweep.pdf`, `figures/s07_best_curves.pdf`, in the report: your protocol, both figures, the ratio table, and the two explanations.

Points (10)

3 prediction · 4 artifacts (protocol followed, equal budgets, all runs logged with settings) · 3 explanation (bias correction; per-parameter scaling).

8 Decoding lab: memorisation versus novelty (10 points)

Goal. Make "hallucination" and "creativity" measurable.

Predict first (write this in `predictions.md` before the agent runs anything)

Which decoding setting will produce the most names that already exist in the training set, which will produce the most duplicate outputs, and which will produce unpronounceable output.

Ask the agent to. Implement greedy decoding, temperature sampling at $T \in \{0.3, 0.5, 0.8, 1.0, 1.5\}$, top-$k$ at $k \in \{3, 10\}$, and nucleus sampling at $p \in \{0.7, 0.9\}$. For each setting generate 500 names from the trained baseline model and report: fraction present in the training set, number of unique outputs, mean length, and the ten most frequent outputs.

Then you. Write a short analysis of the plausibility–diversity trade-off using the numbers. Explain why greedy decoding from `BOS` can only ever produce one name. Relate your findings to Holtzman et al. (2020).

Artifacts for this section

`code/s08/`, `results/s08/decoding.csv` (one row per setting), `results/s08/samples_setting.txt`, in the report: the table, the analysis, and the greedy explanation.

Points (10)

3 prediction · 4 artifacts (all ten settings, 500 samples each, metrics computed correctly against the training split) · 3 explanation.

9 A miniature scaling study (10 points)

Goal. See the shape of scaling laws at a scale where every run is affordable.

Predict first (write this in `predictions.md` before the agent runs anything)

Whether doubling width or doubling depth will help more at this scale; whether the largest model will be the best after 2,000 single-document steps; and roughly how long the largest run will take.

Ask the agent to. Run a grid over `n_embd` $\in \{8, 16, 32, 64\}$ and `n_layer` $\in \{1, 2, 4\}$ for 2,000 steps each, recording parameter count, final trailing-100 loss, and wall-clock time. Estimate training FLOPs per run as $6 \times$ parameters $\times$ tokens processed. Plot loss against parameters and loss against estimated FLOPs, both on log axes.

Then you. Identify which runs are undertrained (loss still falling steeply at the end) and justify the criterion you used. Write two paragraphs connecting the plots to Kaplan et al. (2020) and to the Chinchilla ratio of roughly 20 tokens per parameter (Hoffmann et al. 2022). Compute how many tokens the largest model in your grid would need to reach that ratio, and how many steps that is at one document per step.

Artifacts for this section

`code/s09/`, `results/s09/grid.csv` (12 rows), `figures/s09_loss_vs_params.pdf`, `figures/s09_loss_vs_flops.pdf`, in the report: the table, both plots, the undertraining criterion, the two paragraphs, and the token calculation.

Points (10)

3 prediction · 4 artifacts (12 runs, FLOP estimate shown, log axes) · 3 explanation.

10 New data, new tokenizer, and a measurement trap (10 points)

Goal. Experience the FAQ item "what if I change the dataset?" and learn why losses are comparable only per character.

Predict first (write this in `predictions.md` before the agent runs anything)

Whether per-token loss will rise or fall as the number of BPE merges grows, and whether that change means the model got better or worse.

Ask the agent to. Train the baseline on a different list of short strings of your choosing (English words, city names, element names; include the file). Then implement a minimal byte-pair encoding over the names with 20, 50 and 100 merges, retrain microgpt with each vocabulary (adjust `block_size` if needed and say so), and report the per-token loss for each.

Then you. Require the agent to also report loss *per character*, computed as the total negative log-likelihood of each document divided by its character count. Present both tables side by side and explain which comparison is honest and why. Inspect the first ten merges and compare them with the bigram counts from Section 3. Comment on how the new dataset changed the loss floor and the samples.

Artifacts for this section

`code/s10/` (BPE implementation, modified trainer), the alternative dataset file, `results/s10/tokenizer_comparison.csv` (per-token and per-character loss for 0, 20, 50, 100 merges), `results/s10/merges.txt`, `results/s10/newdata_samples.txt`, in the report: both tables, the merge list, and the explanations.

Points (10)

3 prediction · 4 artifacts (working BPE, consistent block size handling, per-character loss computed correctly) · 3 explanation (honest comparison; merges vs bigram counts).

Submission checklist

Before uploading, confirm every line:

- ☐ `report.pdf` uploaded on its own *and* included in the zip; subsections numbered 1–10 in order.
- ☐ `predictions.md` contains ten dated entries, each written before the matching runs.
- ☐ `code/s01` through `code/s10` each run from the command line as documented in `README.md`.
- ☐ `results/` contains every CSV and sample file named in the "Artifacts" boxes above.
- ☐ `figures/` contains every figure referenced in the report, with axis labels and legends.
- ☐ `transcripts/s01.md` through `transcripts/s10.md` are complete and unedited.
- ☐ Every number in the report is accompanied by the settings (seed, steps, hyperparameters) that produced it.
- ☐ `README.md` names the agent and model used and the total wall-clock time of your runs.
- ☐ The zip is named `lab2_cnetid.zip` and unpacks into a single folder of that name.

Points summary

Section	Title	Points
1	Reproduce, then instrument the loss	10
2	Bug hunt with a gradient checker you design	10
3	The bigram baseline and what attention adds	10
4	The ablation ladder	10
5	Look inside attention	10
6	Port it to tensors and prove equivalence	10
7	Optimizer bake-off with a fair protocol	10
8	Decoding lab: memorisation versus novelty	10
9	A miniature scaling study	10
10	New data, new tokenizer, and a measurement trap	10
	Total	100

Each section: 3 points prediction, 4 points artifacts, 3 points explanation. Late submissions are handled according to the course syllabus.

References

Hoffmann, J. et al. (2022). "Training Compute-Optimal Large Language Models". In: *Advances in Neural Information Processing Systems 35 (NeurIPS)*. arXiv:2203.15556; the "Chinchilla" scaling laws.

Holtzman, A., J. Buys, L. Du, M. Forbes, and Y. Choi (2020). "The Curious Case of Neural Text Degeneration". In: *International Conference on Learning Representations (ICLR)*. arXiv:1904.09751; nucleus (top-p) sampling.

Kaplan, J. et al. (2020). "Scaling Laws for Neural Language Models". In: *arXiv preprint arXiv:2001.08361*.

Karpathy, A. (Feb. 2026a). *microgpt*. Blog post; the primary source for this course. URL: https://karpathy.github.io/2026/02/12/microgpt/ (visited on 10/11/2026).

Karpathy, A. (2026b). *microgpt.py: the most atomic way to train and run inference for a GPT in pure, dependency-free Python*. GitHub gist, 200 lines. URL: https://gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95 (visited on 10/11/2026).